Nov 3, 2026, 6:41 PM

You

so i will share u the code and the problem statement of practical test you just tell everything for viva and the full detail and bsics explaination of code ok

ChatGPT

Absolutely! Send me the practical problem statement and your code, and I'll prepare you completely for the viva.

I'll explain everything assuming you have zero prior knowledge, using simple language that you can understand and confidently explain to your examiner.

For every practical, I'll cover:

1. Understanding the problem statement

- What exactly is the practical asking?
- What is the main objective?
- Which algorithm/model/technology are we using?
- Why are we using it?

2. Basic concepts and theory

- Explain every important concept from scratch.
- Definitions in simple language.
- Real-world examples.
- Why this particular algorithm or model is required.

3. Complete code explanation

I'll go through your code line by line:

- What does each library do?
- Why are we importing it?
- What does each variable represent?
- What does each function do?
- Why are we using particular parameters?
- What happens when that line executes?

4. Working of the practical

- Step-by-step execution flow.
- How input is processed.
- How the model/algorithm works internally.
- How output is generated.
- Explain graphs, accuracy, loss, and other results wherever applicable.

5. Important viva questions

I'll prepare questions in three difficulty levels:

- Easy: Basic definitions and fundamentals.
- Medium: Questions about code, functions, parameters and working.
- Hard: Why questions, comparisons, limitations and deeper concepts.

I'll also provide answers that you can speak directly in your viva.

6. Most important examiner questions

I'll specifically cover:

- Why did you use this algorithm?
- What happens if you change this parameter?
- Can you use another algorithm?
- What is the difference between these concepts?
- What is the expected output?
- What are the limitations?

7. Quick revision before the practical

At the end, I'll give you:

- Important definitions.
- Key points to memorize.
- Important code lines.
- 10–15 most expected viva questions.
- A short explanation of the entire practical that you can speak in front of the examiner.

One important thing: I won't just give you answers to memorize. I'll make sure you understand why something is happening so that even if the examiner asks an unexpected question, you can answer

confidently.

Send me your first practical's problem statement + complete code. Let's prepare one practical at a time! 💪

You

1A. Implement Feedforward Neural Network (FNN) with Keras and TensorFlow for classification of hand-written MNIST dataset using below steps: a. Import the necessary packages b. Load the training and testing data c. Define the network architecture using Keras d. Train the model using SGD with 11 epochs e. Evaluate the network f. Plot the training loss and accuracy

```
# 1. Import libraries
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt

# 2. Load dataset
train = pd.read_csv("mnist_train.csv")
test = pd.read_csv("mnist_test.csv")

x_train = train.drop("label", axis=1).values / 255.0
y_train = train["label"].values

x_test = test.drop("label", axis=1).values / 255.0
y_test = test["label"].values

# 3. Create FNN model
model = tf.keras.Sequential([ tf.keras.layers.Flatten(input_shape=(784,)),
tf.keras.layers.Dense(128, activation='relu'), tf.keras.layers.Dense(10, activation='softmax')
])

# 4. Compile model
model.compile( optimizer='sgd', loss='sparse_categorical_crossentropy', metrics=['accuracy']
)

# 5. Train model
history = model.fit(x_train, y_train, epochs=11)

# 6. Evaluate model
loss, accuracy = model.evaluate(x_test, y_test)
print("Accuracy:", accuracy * 100)

# 7. Plot graphs
```

plt.plot(history.history['loss'], label='Loss')
plt.plot(history.history['accuracy'], label='Accuracy')
plt.legend()
plt.show()...

ChatGPT

$$28 \times 28 = 784 \text{ pixels}$$

You

1C. Build Feedforward Neural Network (FNN) with Keras and TensorFlow for classification of CIFAR10 image dataset using the following steps: a. Import the necessary packages b. Load the training and testing data c. Define the network architecture using Keras d. Train the model using SGD/Adam optimizer e. Evaluate the network f. Plot the training loss and accuracy# 1. Import libraries
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

2. Load CIFAR-10 dataset
data = pd.read_csv("cifar-10.csv")

3. Separate images and labels
X = data.drop("label", axis=1).values
y = data["label"].values

4. Normalize pixel values
X = X / 255.0

5. Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42
)

6. Create FNN model
model = tf.keras.Sequential([tf.keras.layers.Input(shape=(3072,)), tf.keras.layers.Dense(128, activation='relu'), tf.keras.layers.Dense(64, activation='relu'), tf.keras.layers.Dense(10, activation='softmax')
])

```
# 7. Compile model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# 8. Train model
history = model.fit(X_train, y_train, epochs=5, batch_size=32, validation_split=0.2)

# 9. Evaluate model
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Accuracy:", accuracy * 100, "%")
print("Test Loss:", loss)

# 10. Plot accuracy and loss
plt.Figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Accuracy")
plt.legend(["Training", "Validation"])
plt.show()

plt.Figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Loss")
plt.legend(["Training", "Validation"])

plt.show()...
```

ChatGPT

$$32 \times 32 \times 3 = 3072$$

You

1E. Build Feedforward Neural Network (FNN) with Keras and TensorFlow for classification of Sign Language MNIST image dataset using the following steps: a. Import the necessary packages b. Load the training and testing data c. Define the network architecture using Keras d. Train the model using SGD/Adam optimizer e. Evaluate the network f. Plot the training loss and accuracy

```
# 1. Import libraries
import pandas as pd
```

```
import tensorflow as tf
import matplotlib.pyplot as plt

# 2. Load dataset
train = pd.read_csv("sign_mnist_train.csv")
test = pd.read_csv("sign_mnist_test.csv")

# 3. Separate images and labels
X_train = train.drop("label", axis=1).values
y_train = train["label"].values.copy()

X_test = test.drop("label", axis=1).values
y_test = test["label"].values.copy()

# Correct labels (J and Z are excluded)
y_train[y_train > 9] -= 1
y_test[y_test > 9] -= 1

# 4. Normalize images
X_train = X_train / 255.0
X_test = X_test / 255.0

# 5. Create FNN model
model = tf.keras.Sequential([ tf.keras.layers.Input(shape=(784,)), tf.keras.layers.Dense(128,
activation='relu'), tf.keras.layers.Dense(64, activation='relu'), tf.keras.layers.Dense(24,
activation='softmax')
])

# 6. Compile model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# 7. Train model
history = model.fit(X_train, y_train, epochs=5, batch_size=32, validation_split=0.2)

# 8. Evaluate model
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Accuracy:", accuracy * 100, "%")
print("Test Loss:", loss)

# 9. Plot accuracy and loss
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'])
```

```
plt.plot(history.history['val_accuracy'])
plt.title("Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend(["Training", "Validation"])

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend(["Training", "Validation"])

plt.show()...
```

ChatGPT

$$26 - 2 = 24 \text{ classes}$$

You

2A. Implement the CNN model for classifying CIFAR10 image dataset by dividing the model into following 4 stages: a. Loading and preprocessing the image data b. Defining the model's architecture c. Training the model d. Estimating the model's performance # 1. Import libraries

```
import pandas as pd
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

# --------------------------------
# STAGE 1: LOAD AND PREPROCESS DATA
# --------------------------------

# Load CSV file
data = pd.read_csv("cifar-10.csv")

# Separate images and labels
x = data.drop("label", axis=1).values
y = data["label"].values
```

```
# Normalize pixel values
x = x / 255.0

# Reshape images into 32x32 RGB images
x = x.reshape(-1, 32, 32, 3)

# Split training and testing data
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split( x, y, test_size=0.2, random_state=42
)

print("Training data:", x_train.shape)
print("Testing data:", x_test.shape)

# ---------------------------------
# STAGE 2: DEFINE CNN MODEL
# ---------------------------------

model = tf.keras.Sequential([

tf.keras.layers.Input(shape=(32, 32, 3)),

tf.keras.layers.Conv2D(32, (3, 3), activation='relu'), tf.keras.layers.MaxPooling2D((2, 2)),

tf.keras.layers.Conv2D(64, (3, 3), activation='relu'), tf.keras.layers.MaxPooling2D((2, 2)),

tf.keras.layers.Flatten(),

tf.keras.layers.Dense(128, activation='relu'),

tf.keras.layers.Dense(10, activation='softmax')
])

# Compile model
model.compile( optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy']
)

# ---------------------------------
# STAGE 3: TRAIN THE MODEL
# ---------------------------------

history = model.fit( x_train, y_train, epochs=5, batch_size=64, validation_split=0.1
)

# ---------------------------------
# STAGE 4: EVALUATE PERFORMANCE
```

```
# --------------------------------

loss, accuracy = model.evaluate(x_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy)

# Plot accuracy
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.title("Model Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

# Plot loss
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title("Model Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()...
```

ChatGPT

$32 \times 32 \times 3$

You

next time give it little bit short and just before explaining give the practical number and problem statement

ChatGPT

Sure! From the next practical onwards, I'll follow this format:

1. Practical number and problem statement – exactly as you provide it.
2. Basic concept – what the practical does.

3. Code explanation – step by step, in simple language.
4. Important viva questions and answers.
5. Short practical explanation – what to speak in front of the examiner.
6. Quick revision – important values and concepts.

I'll keep it shorter and easier to revise, without skipping important concepts.

You

2B. Implement the Image classification CNN model for classifying hand-written MNIST dataset by dividing the model into following 4 stages: a. Loading and preprocessing the image data b. Defining the model's architecture c. Training the model d. Estimating the model's performance

```
# ------------------------------------------------
# PRACTICAL 2B: CNN FOR HANDWRITTEN MNIST
# ------------------------------------------------

# STAGE 1: Loading and preprocessing data

# 1. Import libraries
import tensorflow as tf
import pandas as pd
import matplotlib.pyplot as plt

# 2. Load local MNIST CSV files
train_data = pd.read_csv("mnist_train.csv")
test_data = pd.read_csv("mnist_test.csv")

# 3. Separate images and labels
x_train = train_data.iloc[:, 1:].values
y_train = train_data.iloc[:, 0].values

x_test = test_data.iloc[:, 1:].values
y_test = test_data.iloc[:, 0].values

# 4. Normalize pixel values
x_train = x_train / 255.0
x_test = x_test / 255.0

# 5. Reshape images for CNN
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# STAGE 2: Define CNN architecture
```

```
model = tf.keras.Sequential([
tf.keras.layers.Input(shape=(28, 28, 1)),
# Extract image features tf.keras.layers.Conv2D(32, (3, 3), activation='relu'),
# Reduce image size tf.keras.layers.MaxPooling2D((2, 2)),
# Convert feature maps into one-dimensional data tf.keras.layers.Flatten(),
# Hidden layer tf.keras.layers.Dense(128, activation='relu'),
# Output layer for 10 digits tf.keras.layers.Dense(10, activation='softmax')
])
# STAGE 3: Compile and train the model
model.compile( optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy']
)
history = model.fit( x_train, y_train, epochs=5, validation_split=0.1
)
# STAGE 4: Evaluate model performance
loss, accuracy = model.evaluate(x_test, y_test)
print("Test Loss:", loss)
print("Test Accuracy:", accuracy * 100, "%")
# Plot accuracy
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
# Plot loss
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
```

plt.show()...

ChatGPT

Adam	Optimizer that updates weights
Sparse categorical crossentropy	Loss function for multiple classes with integer labels
Accuracy	Measures correct predictions

You

2C. Implement the CNN model for classifying Sign Language MNIST by dividing the model into following 4 stages: a. Loading and preprocessing the image data b. Defining the model's architecture c. Training the model d. Estimating the model's performance
PRACTICAL 2C: CNN FOR SIGN LANGUAGE MNIST

```
# 1. Import libraries
import tensorflow as tf
import pandas as pd
import matplotlib.pyplot as plt

# 2. Load offline dataset
train_data = pd.read_csv("sign_mnist_train.csv")
test_data = pd.read_csv("sign_mnist_test.csv")

# 3. Separate images and labels
x_train = train_data.iloc[:, 1:].values
y_train = train_data.iloc[:, 0].values

x_test = test_data.iloc[:, 1:].values
y_test = test_data.iloc[:, 0].values

# 4. Normalize images
x_train = x_train / 255.0
x_test = x_test / 255.0

# 5. Reshape images for CNN
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# 6. Create CNN model
model = tf.keras.Sequential([ tf.keras.layers.Input(shape=(28, 28, 1)),
```

```
tf.keras.layers.Conv2D(32, (3, 3), activation='relu'),

tf.keras.layers.MaxPooling2D((2, 2)),

tf.keras.layers.Flatten(),

tf.keras.layers.Dense(128, activation='relu'),

tf.keras.layers.Dense(25, activation='softmax')
])

# 7. Compile model
model.compile( optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy']
)

# 8. Train model
history = model.fit( x_train, y_train, epochs=5, validation_split=0.1
)

# 9. Evaluate model
loss, accuracy = model.evaluate(x_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy)

# 10. Plot accuracy
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

# 11. Plot loss
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()...
```

ChatGPT

Adam	Optimizer that updates model weights
Sparse categorical crossentropy	Loss function for multiple classes with integer labels
Accuracy	Measures correct predictions

You

3A. Implement credit card fraud detection for given credit card fraud detection Kaggle dataset using Autoencoder and build the model by using the following steps: • Dataset: The dataset contains transactions labeled as fraudulent or non-fraudulent, with a severe class imbalance. • Goal: Identify fraudulent transactions by training an autoencoder only on non fraudulent transactions. Steps: • Preprocess the data by scaling features (features are already PCA-transformed, so they're anonymized). • Train an autoencoder on non-fraudulent transactions to learn typical spending patterns. • Calculate reconstruction errors for transactions and set a threshold to detect fraud. • Compile the models with Optimizer, Loss, and Evaluation Metrics # 1. Import libraries

```
import pandas as pd
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

# STEP 1: Load dataset
data = pd.read_csv("creditcard.csv")

# Separate normal transactions
normal_data = data[data["Class"] == 0]

# Separate features and labels
X = data.drop("Class", axis=1).values
y = data["Class"].values

# Use only normal transactions for training
x_train = normal_data.drop("Class", axis=1).values

# Normalize data
mean = x_train.mean(axis=0)
std = x_train.std(axis=0)
std[std == 0] = 1

x_train = (x_train - mean) / std
```

```
X = (X - mean) / std

# STEP 2: Build Autoencoder
model = tf.keras.Sequential([ tf.keras.layers.Input(shape=(X.shape[1],)),

# Encoder tf.keras.layers.Dense(16, activation='relu'), tf.keras.layers.Dense(8, activation='relu'),

# Decoder tf.keras.layers.Dense(16, activation='relu'), tf.keras.layers.Dense(X.shape[1],
activation='linear')
])

# Compile model
model.compile(optimizer='adam', loss='mse')

# STEP 3: Train model
history = model.fit( x_train, x_train, epochs=10, batch_size=256, validation_split=0.2
)

# STEP 4: Detect fraud

# Reconstruct transactions
reconstructed = model.predict(X)

# Calculate reconstruction error
errors = np.mean(np.square(X - reconstructed), axis=1)

# Set threshold
threshold = np.percentile(errors, 95)

# Predict fraud
predictions = (errors > threshold).astype(int)

# STEP 5: Evaluate model

accuracy = np.mean(predictions == y)

print("Accuracy:", accuracy * 100, "%")

# Plot training loss
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title("Autoencoder Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()...
```

ChatGPT

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(X_i - \hat{X}_i)^2$$

You

3B. Implement Industrial Equipment Anomaly Detection on the MVTec AD Kaggle dataset using Autoencoder and build the model by using the following steps: Steps: • Preprocess images (resize, normalize) for input into a convolutional autoencoder. • Build a convolutional autoencoder (CNN-based encoder-decoder) to learn the representations of normal object images. • Use reconstruction errors in pixel values to detect anomalies in test images. Dataset: MVTec AD is an anomaly detection dataset for industrial defect detection, containing images of normal and defective objects. Goal: Detect defects in objects by training an image-based autoencoder on images of defect-free objects.

```
# Practical 3B: Industrial Equipment Anomaly Detection
# Using Convolutional Autoencoder on MVTec AD Dataset

# STEP 1: Import libraries

import pandas as pd
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

# STEP 2: Load ECG dataset

data = pd.read_csv("ecg.csv", header=None)

# Separate ECG signals and labels
X = data.iloc[:, :-1].values
y = data.iloc[:, -1].values

print("Dataset shape:", X.shape)

# STEP 3: Select only normal ECG signals

normal_label = 1

X_normal = X[y == normal_label]

# Normalize the data
```

```
mean = X_normal.mean()
std = X_normal.std()

X_normal = (X_normal - mean) / std
X_all = (X - mean) / std

# Reshape for Conv1D
X_normal = X_normal.reshape(X_normal.shape[0], X_normal.shape[1], 1)
X_all = X_all.reshape(X_all.shape[0], X_all.shape[1], 1)

# Split normal data into training and validation
X_train = X_normal[:int(0.8 * len(X_normal))]
X_val = X_normal[int(0.8 * len(X_normal)):]

# STEP 4: Build 1D Convolutional Autoencoder

model = tf.keras.Sequential([

tf.keras.layers.Input(shape=(X_train.shape[1], 1)),

# Encoder tf.keras.layers.Conv1D(32, 3, activation='relu', padding='same'),
tf.keras.layers.MaxPooling1D(2, padding='same'),

tf.keras.layers.Conv1D(16, 3, activation='relu', padding='same'),
tf.keras.layers.MaxPooling1D(2, padding='same'),

# Decoder tf.keras.layers.Conv1D(16, 3, activation='relu', padding='same'),
tf.keras.layers.UpSampling1D(2),

tf.keras.layers.Conv1D(32, 3, activation='relu', padding='same'),
tf.keras.layers.UpSampling1D(2),

tf.keras.layers.Conv1D(1, 3, activation='linear', padding='same')
])

# STEP 5: Compile the model

model.compile(optimizer='adam', loss='mse')

# STEP 6: Train using normal ECG signals

history = model.fit( X_train, X_train, epochs=10, batch_size=32, validation_data=(X_val, X_val)
)

# STEP 7: Calculate reconstruction error

reconstructed = model.predict(X_all)

errors = np.mean(np.square(X_all - reconstructed), axis=(1, 2))
```

```
# Set threshold using normal validation data
val_reconstructed = model.predict(X_val)

val_errors = np.mean( np.square(X_val - val_reconstructed), axis=(1, 2)
)

threshold = np.percentile(val_errors, 95)

print("Anomaly threshold:", threshold)

# STEP 8: Detect anomalies

predictions = (errors > threshold).astype(int)

# Actual labels: normal = 0, abnormal = 1
actual = (y != normal_label).astype(int)

accuracy = np.mean(predictions == actual) * 100

print("Accuracy:", accuracy, "%")

# STEP 9: Plot training loss

plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("Autoencoder Training Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

# STEP 10: Display anomaly detection results

plt.hist(errors, bins=50)
plt.axvline(threshold, color='red', label='Threshold')
plt.title("ECG Reconstruction Errors")
plt.xlabel("Reconstruction Error")
plt.ylabel("Number of Samples")
plt.legend()
plt.show()

# STEP 11: Save model

model.save("ecg_autoencoder.keras")

print("Model saved successfully!")...
```

ChatGPT

MVTec AD image dataset	ECG signal dataset
Industrial defect detection	ECG anomaly detection
Conv2D	Conv1D
MaxPooling2D	MaxPooling1D
UpSampling2D	UpSampling1D
Image reconstruction	Signal reconstruction
ecg.csv	CSV-based ECG data

You

3C. Implement Anomaly Detection in Energy Consumption Data (Smart Grid Dataset) using Autoencoder and build the model by using the following steps: Steps: • Preprocess the data to fill any missing values, normalize, and create time windows for sequential input. • Use an LSTM-based autoencoder to learn temporal patterns in normal consumption. • Set an error threshold and detect anomalies when the reconstruction error exceeds this threshold. Dataset: The Smart Grid dataset records electricity consumption across multiple timestamps for various households. Goal: Detect unusual consumption patterns by training an autoencoder on typical (normal) energy usage patterns # STEP 1: Import libraries

```
import pandas as pd
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler

# STEP 2: Load dataset

data = pd.read_csv("energy consumption.csv")

print(data.head())
print("Dataset shape:", data.shape)

# STEP 3: Preprocess data

# Convert timestamp into date format
data["Timestamp"] = pd.to_datetime( data["Timestamp"], format="%d-%m-%Y %H:%M"
)

# Sort data according to timestamp
data = data.sort_values("Timestamp")
```

```
# Fill missing values
data = data.ffill().bfill()

# Select input features
features = [ "Electricity_Consumed", "Temperature", "Humidity", "Wind_Speed",
"Avg_Past_Consumption"
]

X = data[features].values

# Store actual labels
y = data["Anomaly_Label"].values

# Normalize data
scaler = MinMaxScaler()
X = scaler.fit_transform(X)

print("Preprocessing completed!")

# STEP 4: Create time windows

window_size = 24

X_window = []
y_window = []

for i in range(len(X) - window_size + 1):

X_window.append(X[i:i + window_size])

# Label of the last reading in the window y_window.append(y[i + window_size - 1])

X_window = np.array(X_window)
y_window = np.array(y_window)

print("Window shape:", X_window.shape)

# STEP 5: Split training and testing data

split = int(0.8 * len(X_window))

X_train = X_window[:split]
X_test = X_window[split:]

y_train = y_window[:split]
y_test = y_window[split:]

# Select only normal windows for training
normal_indices = []
```

```
for i in range(len(X_train)):
if np.all(y_train[i] == 0): normal_indices.append(i)
X_train = X_train[normal_indices]
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
# STEP 6: Build LSTM Autoencoder
model = tf.keras.Sequential([
# Encoder tf.keras.layers.Input(shape=(window_size, 5)),
tf.keras.layers.LSTM(64, activation="tanh"),
# Compressed representation tf.keras.layers.RepeatVector(window_size),
# Decoder tf.keras.layers.LSTM( 64, activation="tanh", return_sequences=True ),
# Reconstruct original features tf.keras.layers.TimeDistributed( tf.keras.layers.Dense(5) )
])
# STEP 7: Compile model
model.compile( optimizer="adam", loss="mse"
)
# STEP 8: Train model
history = model.fit( X_train, X_train, epochs=10, batch_size=32, validation_split=0.1
)
# STEP 9: Calculate reconstruction error
train_output = model.predict(X_train)
train_error = np.mean( np.square(X_train - train_output), axis=(1, 2)
)
# Set threshold
threshold = np.percentile(train_error, 95)
print("Anomaly Threshold:", threshold)
# STEP 10: Detect anomalies in test data
test_output = model.predict(X_test)
test_error = np.mean( np.square(X_test - test_output), axis=(1, 2)
```

```
)

# Predict anomalies
predictions = (test_error > threshold).astype(int)

print("Anomalies detected:", np.sum(predictions))

# STEP 11: Compare actual and predicted results

actual = (y_test != 0).astype(int)

accuracy = np.mean(predictions == actual) * 100

print("Accuracy:", accuracy, "%")

# STEP 12: Plot training loss

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.title("LSTM Autoencoder Training Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

# STEP 13: Plot anomaly detection

plt.figure(figsize=(10, 5))

plt.plot(test_error, label="Reconstruction Error")

plt.axhline( threshold, color="red", label="Anomaly Threshold"
)

plt.title("Energy Consumption Anomaly Detection")
plt.xlabel("Time Window")
plt.ylabel("Reconstruction Error")
plt.legend()
plt.show()

# STEP 14: Save model

model.save("energy_autoencoder.keras")

print("Model saved successfully!")...
```

ChatGPT

Input(24,5)	24 readings and 5 features
LSTM(64)	Learns time-based patterns
RepeatVector(24)	Repeats compressed information 24 times
LSTM(64, return_sequences=True)	Reconstructs the sequence
TimeDistributed(Dense(5))	Produces 5 features for every time step

You

3D. Detect the faults in Sensor Data from Machinery (NASA Turbofan Engine Dataset) using Autoencoder and build the model by using the following steps: • Dataset: The dataset contains sensor readings from turbofan engines with labels indicating whether a reading is faulty. • Goal: Detect machinery faults by training an autoencoder on sensor data from healthy engines. Steps: • Preprocess data by scaling sensor readings. • Use an LSTM-based autoencoder (to capture sequential data) to learn patterns in normal sensor readings. • Calculate reconstruction error for test data, with a threshold to classify high-error instances as faults.

```
# 1. Import libraries
import pandas as pd
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler

# 2. Load dataset
data = pd.read_csv("NASA TurboFan Engine.csv")

# 3. Sort data by engine and cycle
data = data.sort_values(["unit", "cycle"])

# 4. Select sensor columns
sensor_columns = [ "sensor" + str(i) for i in range(1, 22)
]

X = data[sensor_columns].values
y = data["Fault_Label"].values

# 5. Normalize sensor values
scaler = MinMaxScaler()
X = scaler.fit_transform(X)
```

```
# 6. Create sequences of 20 readings
sequence_length = 20

X_sequences = []
y_sequences = []

# Create sequences separately for each engine
for unit in data["unit"].unique():

engine_data = data[data["unit"] == unit]

engine_X = scaler.transform( engine_data[sensor_columns].values )

engine_y = engine_data["Fault_Label"].values

for i in range(len(engine_X) - sequence_length + 1):

X_sequences.append( engine_X[i:i + sequence_length] )

# Label the sequence using its last reading y_sequences.append( engine_y[i +
sequence_length - 1] )

X_sequences = np.array(X_sequences)
y_sequences = np.array(y_sequences)

print("Sequence data shape:", X_sequences.shape)

# 7. Split healthy and faulty data
X_train = X_sequences[y_sequences == 0]

print("Healthy training data:", X_train.shape)

# 8. Build LSTM Autoencoder
model = tf.keras.Sequential([ tf.keras.layers.Input(shape=(20, 21)),

tf.keras.layers.LSTM(64),

tf.keras.layers.RepeatVector(20),

tf.keras.layers.LSTM(64, return_sequences=True),

tf.keras.layers.TimeDistributed( tf.keras.layers.Dense(21) )
])

# 9. Compile model
model.compile( optimizer="adam", loss="mse"
)

# 10. Train the model using healthy data
```

```
history = model.fit( X_train, X_train, epochs=10, batch_size=32, validation_split=0.1
)

# 11. Reconstruct all sequences
reconstructed = model.predict(X_sequences)

# 12. Calculate reconstruction error
errors = np.mean( np.square(X_sequences - reconstructed), axis=(1, 2)
)

# 13. Set fault threshold
threshold = np.percentile(errors, 95)

print("Fault Threshold:", threshold)

# 14. Predict faults
predictions = (errors > threshold).astype(int)

# 15. Calculate accuracy
accuracy = np.mean(predictions == y_sequences)

print("Fault Detection Accuracy:", accuracy * 100, "%")

# 16. Plot training loss
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Model Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

# 17. Plot reconstruction errors
plt.plot(errors, label="Reconstruction Error")
plt.axhline( y=threshold, color="red", linestyle="--", label="Fault Threshold"
)
plt.title("Fault Detection using LSTM Autoencoder")
plt.xlabel("Sequence Number")
plt.ylabel("Reconstruction Error")
plt.legend()
plt.show()

# 18. Save model
```

model.save("nasa_turbofan_autoencoder.keras")

print("Model saved successfully!")...

ChatGPT

Input(20,21)	20 readings with 21 sensors each
LSTM(64)	Learns and compresses sequential patterns
RepeatVector(20)	Repeats compressed information 20 times
LSTM(64, return_sequences=True)	Reconstructs the sequence
TimeDistributed(Dense(21))	Produces 21 sensor values at each time step

Nov 3, 2026, 9:46 PM

You

3E. Use Autoencoder to implement Network Intrusion Detection on NSL-KDD dataset. Build the model • Dataset: NSL-KDD is a network intrusion detection dataset with various types of network traffic, labeled as either "normal" or "attack." • Goal: Identify network intrusions by training an autoencoder on normal network traffic data, then flagging traffic as anomalous if the reconstruction error is high. Steps: • Preprocess the NSL-KDD dataset, particularly by encoding categorical features and scaling numerical features. • Define an autoencoder similar to the ECG example, focusing on the reconstruction of normal network patterns. • Train on normal data and set an error threshold to detect intrusions.

1. Import libraries
import pandas as pd
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler

2. Load ECG dataset
data = pd.read_csv("ecg.csv", header=None)

3. Separate ECG signals and labels
X = data.iloc[:, :-1].values
y = data.iloc[:, -1].values

print("Dataset shape:", X.shape)

```
# 4. Normalize ECG signals
scaler = MinMaxScaler()

X = scaler.fit_transform(X)

# 5. Select only normal ECG signals for training
X_train = X[y == 1]

print("Normal training data:", X_train.shape)

# 6. Build Autoencoder
model = tf.keras.Sequential([ tf.keras.layers.Input(shape=(X.shape[1],)),

tf.keras.layers.Dense(32, activation="relu"),

tf.keras.layers.Dense(16, activation="relu"),

tf.keras.layers.Dense(32, activation="relu"),

tf.keras.layers.Dense(X.shape[1], activation="sigmoid")
])

# 7. Compile model
model.compile( optimizer="adam", loss="mse"
)

# 8. Train using normal ECG signals
history = model.fit( X_train, X_train, epochs=10, batch_size=32, validation_split=0.2
)

# 9. Reconstruct ECG signals
reconstructed = model.predict(X)

# 10. Calculate reconstruction error
errors = np.mean( np.square(X - reconstructed), axis=1
)

# 11. Set threshold
threshold = np.percentile(errors[y == 1], 95)

print("Threshold:", threshold)

# 12. Detect abnormal ECG signals
predictions = (errors > threshold).astype(int)

# Actual labels: normal = 0, abnormal = 1
actual = (y != 1).astype(int)
```

```
# 13. Calculate accuracy
accuracy = np.mean(predictions == actual)

print("Detection Accuracy:", accuracy * 100, "%")

# 14. Plot training loss
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.title("Autoencoder Training Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

# 15. Plot reconstruction errors
plt.plot(errors, label="Reconstruction Error")

plt.axhline( y=threshold, color="red", linestyle="--", label="Threshold"
)

plt.title("ECG Anomaly Detection")
plt.xlabel("ECG Sample")
plt.ylabel("Error")
plt.legend()
plt.show()

# 16. Save model
model.save("ecg_autoencoder.keras")

print("Model saved successfully!")...
```

ChatGPT

ecg.csv	NSL-KDD train/test files
ECG signals	Network traffic records
Numerical signal values	Numerical + categorical features
No categorical encoding	One-hot encoding required
ECG anomaly detection	Network intrusion detection

You

3F. Using the Sign Language MNIST dataset, select any one of the 24 alphabet classes (label Y-24) as an anomaly. Train an Autoencoder–Decoder model using the remaining 23 classes and detect the selected class as an anomaly based on reconstruction error. # 1. Import libraries

```
import pandas as pd
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

# ----------------------------------------
# STEP 1: LOAD AND PREPROCESS DATA
# ----------------------------------------

# Load dataset
train_data = pd.read_csv("sign_mnist_train.csv")
test_data = pd.read_csv("sign_mnist_test.csv")

# Separate images and labels
x_train = train_data.drop("label", axis=1).values
y_train = train_data["label"].values

x_test = test_data.drop("label", axis=1).values
y_test = test_data["label"].values

# Normalize pixel values
x_train = x_train / 255.0
x_test = x_test / 255.0

# Reshape images
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# Select Y (label 24) as anomaly
anomaly_label = 24

# Keep only 23 normal classes for training
x_normal = x_train[y_train != anomaly_label]

print("Normal training images:", len(x_normal))
print("Anomaly class: Y (label 24)")

# ----------------------------------------
# STEP 2: BUILD AUTOENCODER MODEL
```

```
# ----------------------------------------

model = tf.keras.Sequential([

# Encoder tf.keras.layers.Input(shape=(28, 28, 1)), tf.keras.layers.Flatten(),
tf.keras.layers.Dense(128, activation="relu"), tf.keras.layers.Dense(64, activation="relu"),

# Decoder tf.keras.layers.Dense(128, activation="relu"), tf.keras.layers.Dense(784,
activation="sigmoid"), tf.keras.layers.Reshape((28, 28, 1))

])

# Compile model
model.compile( optimizer="adam", loss="mse"
)

# ----------------------------------------
# STEP 3: TRAIN AUTOENCODER
# ----------------------------------------

history = model.fit( x_normal, x_normal, epochs=10, batch_size=128, validation_split=0.1
)

# ----------------------------------------
# STEP 4: CALCULATE RECONSTRUCTION ERROR
# ----------------------------------------

# Reconstruct test images
reconstructed = model.predict(x_test)

# Calculate reconstruction error
errors = np.mean( np.square(x_test - reconstructed), axis=(1, 2, 3)
)

# Calculate threshold using normal test images
normal_errors = errors[y_test != anomaly_label]

threshold = np.percentile(normal_errors, 95)

print("Threshold:", threshold)

# ----------------------------------------
# STEP 5: DETECT ANOMALIES
# ----------------------------------------

# Predict anomaly
predictions = (errors > threshold).astype(int)
```

```
# Actual labels: 1 = anomaly, 0 = normal
actual = (y_test == anomaly_label).astype(int)

# Calculate accuracy
accuracy = np.mean(predictions == actual)

print("Detection Accuracy:", accuracy)

# Count detected anomalies
print("Actual anomalies:", np.sum(actual))
print("Detected anomalies:", np.sum(predictions))

# ----------------------------------------
# STEP 6: DISPLAY RESULTS
# ----------------------------------------

# Display original and reconstructed image
plt.figure(figsize=(8, 4))

plt.subplot(1, 2, 1)
plt.imshow(x_test[0].reshape(28, 28), cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(reconstructed[0].reshape(28, 28), cmap="gray")
plt.title("Reconstructed Image")
plt.axis("off")

plt.show()

# Plot training loss
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.title("Autoencoder Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()...
```

ChatGPT

Input	Receives 28 × 28 image
Flatten	Converts image into 784 values
Dense(128)	Learns image features
Dense(64)	Compresses information
Dense(128)	Starts reconstruction
Dense(784)	Reconstructs pixel values
Reshape	Converts values back to an image

You

4A. Implement the CBOW for document classification using deep learning. Steps (for Implementing CBOW for Document Classification): 1. Data Preparation: You need a labeled dataset for document classification. You can use datasets like the IMDb movie reviews dataset (for sentiment classification) or 20 Newsgroups dataset (for topic classification). Load and Preprocess the data: • from tensorflow.keras.datasets import imdb # For loading • Pad sequences to ensure uniform input length • Convert the IMDB dataset to a list of sentences (each sentence is a list of word) • Tokenize the text into words. • Lowercase the text and remove stop words. • Create a vocabulary from the training data. 2. Define CBOW Model: CBOW learns to predict a target word based on surrounding context words. For document classification, you need to modify the CBOW model to work with document-level contexts. In CBOW, the context consists of words surrounding the target word, but for documents, you will use the embeddings from 3. Model Architecture: all the words in the document. • Embedding Layer: First, use the CBOW architecture to learn word embeddings from the text corpus. • Document Representation: After learning the word embeddings, represent a document by averaging the embeddings of all words in the document. • Fully Connected Layer: Use this document representation as input to a neural network (e.g., a dense layer) for classification. • Train word2vec model with CBOW (sg=0 for CBOW) • Convert words to embeddings • Build the feed forward neural network for document classification 4. Evaluate the Model on test set using loss and accuracy.

Practical 4A: CBOW for Document Classification

```
# 1. Import libraries
import pandas as pd
import numpy as np
import re
import matplotlib.pyplot as plt
from gensim.models import Word2Vec
```

```
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input

# 2. Load dataset
data = pd.read_csv("IMDB Dataset.csv")

# Convert sentiment into numbers
data = data.dropna(subset=["review", "sentiment"])
labels = data["sentiment"].map({"positive": 1, "negative": 0}).values

# 3. Preprocess reviews
stop_words = {"the", "is", "a", "an", "and", "to", "of", "in", "it", "this", "that", "was", "for"}

def preprocess(text): text = re.sub("[^a-z ]", "", text.lower()) return [word for word in text.split()
if word not in stop_words]

sentences = data["review"].astype(str).apply(preprocess).tolist()

# 4. Split dataset
X_train, X_test, y_train, y_test = train_test_split( sentences, labels, test_size=0.2,
random_state=42
)

# 5. Train CBOW Word2Vec model
cbow_model = Word2Vec( sentences=X_train, vector_size=100, window=5, min_count=2, sg=0,
epochs=10
)

print("CBOW model trained successfully!")

# 6. Convert documents into average word embeddings
def get_document_vector(words): vectors = [cbow_model.wv[w] for w in words if w in
cbow_model.wv]

if len(vectors) == 0: return np.zeros(100)

return np.mean(vectors, axis=0)

X_train_vectors = np.array([get_document_vector(x) for x in X_train])
X_test_vectors = np.array([get_document_vector(x) for x in X_test])

# 7. Build classification model
model = Sequential([ Input(shape=(100,)), Dense(64, activation="relu"), Dense(1,
activation="sigmoid")
```

```
])

# 8. Compile model
model.compile( optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"]
)

# 9. Train model
history = model.fit( X_train_vectors, y_train, epochs=10, batch_size=32, validation_split=0.2
)

# 10. Evaluate model
loss, accuracy = model.evaluate(X_test_vectors, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy)

# 11. Plot accuracy and loss
plt.plot(history.history["accuracy"], label="Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.legend()
plt.title("Model Accuracy")
plt.show()

plt.plot(history.history["loss"], label="Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.legend()
plt.title("Model Loss")
plt.show()

# 12. Test a sample review
sample = preprocess("This movie was amazing and I really enjoyed it")
sample_vector = get_document_vector(sample)

prediction = model.predict(np.array([sample_vector]), verbose=0)

print("Sentiment:", "Positive" if prediction[0][0] >= 0.5 else "Negative")...
```

ChatGPT

review	Movie review text
sentiment	Positive or negative

You

4B. Implement the Continuous Bag of Words (CBOW) Model for the twitter dataset (download from Kaggle) using the below steps: a. Data Collection and Preprocessing: Use Kaggle Twitter dataset or any dataset with tweets. Tokenize the tweets, clean the text, and prepare it for training. b. Train Word2Vec CBOW Model c. Evaluate Results: After training, we'll check the learned word embeddings and possibly evaluate using some examples.

Practical 4B: CBOW Model for Twitter Dataset

```
# 1. Import libraries
import pandas as pd
import re
from gensim.models import Word2Vec

# 2. Load dataset
data = pd.read_csv("twitter_dataset.csv")

print("Dataset Columns:", data.columns.tolist())

# 3. Select tweet column automatically
possible_columns = ["text", "tweet", "tweets", "content"]

tweet_column = next( (col for col in data.columns if col.lower() in possible_columns),
data.columns[-1]
)

tweets = data[tweet_column].dropna()

print("Selected Column:", tweet_column)

# 4. Clean and tokenize tweets
sentences = []

for tweet in tweets: tweet = re.sub(r"[^a-z\s]", "", str(tweet).lower()) words = tweet.split()

if words: sentences.append(words)

# 5. Check processed data
print("Total Tweets:", len(sentences))

if not sentences: raise ValueError("No valid tweets found. Check your dataset column.")

print("Sample Tweet:", sentences[0])

# 6. Train CBOW Word2Vec model
model = Word2Vec( sentences=sentences, vector_size=100, window=5, min_count=1, sg=0,
epochs=10
```

)

print("\nCBOW Model Trained Successfully!")

7. Display word embedding
word = list(model.wv.key_to_index.keys())[0]

print("\nWord:", word)
print("Word Embedding:", model.wv[word])

8. Find similar words
print("\nSimilar Words:")
print(model.wv.most_similar(word))

9. Save model
model.save("cbow_twitter.model")

print("\nModel Saved Successfully!")...

ChatGPT

sentences	Cleaned tweets
vector_size=100	Each word gets 100 numbers (embedding)
window=5	Considers up to 5 surrounding words
min_count=1	Keeps words appearing at least once
sg=0	Selects CBOW
epochs=10	Trains the model 10 times over the data

You

//explain the logic of it and explain the code whats happening

Practical 4C: CBOW Model for Textual Document

1. Import libraries
import re
from gensim.models import Word2Vec

2. Read document
with open("document.txt", "r", encoding="utf-8") as file: text = file.read()

3. Clean and tokenize text
text = re.sub(r"[^a-z\s]", "", text.lower())

```
words = text.split()

# Word2Vec needs a list of sentences
sentences = [words]

print("Total Words:", len(words))
print("Sample Words:", words[:20])

if not words: raise ValueError("Document is empty!")

# 4. Train CBOW model
model = Word2Vec( sentences=sentences, vector_size=100, window=5, min_count=1, sg=0,
epochs=100
)

print("\nCBOW Model Trained Successfully!")

# 5. Display word embedding
word = words[0]

print("\nSelected Word:", word)
print("Word Embedding:", model.wv[word])

# 6. Find similar words
print("\nSimilar Words:")
print(model.wv.most_similar(word))

# 7. Save model
model.save("cbow_document.model")

print("\nModel Saved Successfully!")...
```

ChatGPT

sentences	Input words for training
vector_size=100	Each word is represented using 100 numbers
window=5	Considers surrounding words within a window of 5
min_count=1	Keeps words appearing at least once
sg=0	Uses CBOW architecture
epochs=100	Trains the model for 100 iterations over the data

You

explain the logic of the code and whats happening what we are doing and viva thory

4D. Building a Recommendation System with CBOW Word Embeddings Problem: Use CBOW word embeddings to build a content-based recommendation system. For example, recommend similar articles or tweets based on the embeddings of keywords in a user's input. Steps: 1. Data Collection and Preprocessing: Gather a dataset (e.g., a collection of tweets, articles, or products) with textual content (descriptions, reviews, etc.). Tokenize the text data, clean the data, and prepare it for training. 2. Train CBOW Model: a. Train the Word2Vec model using CBOW to learn word embeddings from the text data or Train the Word2Vec CBOW model on a dataset of articles or tweets. b. For a given article (or tweet), calculate the average of the embeddings of all the words in the document. c. For each document in the dataset, calculate the cosine similarity between the document's average word embedding and the user's input embedding. d. Document Representation: Represent each document (e.g., tweet, product description, article) as a vector by averaging the embeddings of all words in the document. 3. Recommendation Logic: For a given input document (e.g., a user's search or a product), recommend the most similar documents using cosine similarity between their vector representations.
Practical 4D: Recommendation System using CBOW

```
# 1. Import libraries
import pandas as pd
import numpy as np
import re
from gensim.models import Word2Vec

# 2. Load dataset
data = pd.read_csv("twitter_dataset.csv")

print("Available Columns:", data.columns.tolist())

# 3. Automatically select tweet column
possible_columns = ["text", "tweet", "tweets", "content", "message"]

text_column = next( (col for col in data.columns if col.strip().lower() in possible_columns),
data.columns[-1]
)

tweets = data[text_column].dropna().astype(str)

print("Selected Column:", text_column)

# 4. Clean and tokenize tweets
```

```
stop_words = {"the", "is", "a", "an", "and", "to", "of", "in", "it", "this", "that", "for", "on", "with", "are", "was", "be"}

sentences = []

for tweet in tweets: tweet = re.sub(r"[^a-z\s]", "", tweet.lower()) words = [w for w in tweet.split() if w not in stop_words]

if words: sentences.append(words)

if not sentences: raise ValueError("No valid tweets found!")

print("Total Tweets:", len(sentences))

# 5. Train CBOW model
cbow_model = Word2Vec( sentences=sentences, vector_size=100, window=5, min_count=1, sg=0, epochs=10
)

print("CBOW Model Trained Successfully!")

# 6. Convert tweets into average word vectors
def get_vector(words): vectors = [cbow_model.wv[w] for w in words if w in cbow_model.wv]

if not vectors: return np.zeros(100)

return np.mean(vectors, axis=0)

tweet_vectors = np.array([get_vector(words) for words in sentences])

# 7. Take user input
user_input = input("Enter your search keywords: ").lower()
user_input = re.sub(r"[^a-z\s]", "", user_input)

input_words = [w for w in user_input.split() if w not in stop_words]
input_vector = get_vector(input_words)

# 8. Calculate cosine similarity
similarities = []

for vector in tweet_vectors: similarity = np.dot(input_vector, vector) /
( np.linalg.norm(input_vector) * np.linalg.norm(vector) + 1e-10 ) similarities.append(similarity)

# 9. Display top 5 recommendations
top_indices = np.argsort(similarities)[::-1][:5]

print("\nRecommended Tweets:")

for i in top_indices: print("\nTweet:", " ".join(sentences[i])) print("Similarity:",
```

round(similarities[i], 4))...

ChatGPT

$$\text{Cosine Similarity} = \frac{A \cdot B}{\|A\| \times \|B\|}$$

You

5A. Implement Object detection using Transfer Learning with CNN for the image classification-caltec-101 dataset and VGG / ResNet pre-trained models. Steps: a. Set up the environment b. Loading and preprocessing data c. Choose the pre-trained Model (Transfer Learning) d. Freeze the base layers and add custom layers for fine tuning e. Object detection training f. Compute loss g. Evaluate the Model h. Visualize the prediction i. Fine Tune the Model: Fine-tuning involves unfreezing the top layers of the pre-trained model and retraining it on the new dataset to adapt it better to your task. # Practical 5A: Transfer Learning using VGG16

```
# 1. Import libraries
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers

# 2. Load dataset
train = keras.utils.image_dataset_from_directory( "caltech-101", image_size=(128, 128),
validation_split=0.2, subset="training", seed=42
)

val = keras.utils.image_dataset_from_directory( "caltech-101", image_size=(128, 128),
validation_split=0.2, subset="validation", seed=42
)

n_classes = len(train.class_names)
class_names = train.class_names

print("Total Classes:", n_classes)

train = train.prefetch(2)
val = val.prefetch(2)

# 3. Load pre-trained VGG16 model
base = keras.applications.VGG16( weights="imagenet", include_top=False, input_shape=(128,
```

```
128, 3)
)

# Freeze base layers
base.trainable = False

# 4. Build classification model
model = keras.Sequential([ layers.Input(shape=(128, 128, 3)),
layers.Lambda(keras.applications.vgg16.preprocess_input), base,
layers.GlobalAveragePooling2D(), layers.Dense(128, activation="relu"), layers.Dense(n_classes,
activation="softmax")
])

# 5. Compile model
model.compile( optimizer="adam", loss="sparse_categorical_crossentropy",
metrics=["accuracy"]
)

model.summary()

# 6. Train model
history = model.fit( train, validation_data=val, epochs=3
)

# 7. Evaluate model
loss, accuracy = model.evaluate(val)

print("Validation Loss:", loss)
print("Validation Accuracy:", accuracy)

# 8. Plot accuracy and loss
plt.plot(history.history["accuracy"], label="Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.legend()
plt.title("Model Accuracy")
plt.show()

plt.plot(history.history["loss"], label="Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.legend()
plt.title("Model Loss")
plt.show()
```

9. Visualize prediction
for images, labels in val.take(1): prediction = model.predict(images[:1], verbose=0) index = prediction[0].argmax()

plt.imshow(images[0].numpy().astype("uint8")) plt.title("Predicted: " + class_names[index]) plt.axis("off") plt.show()

10. Fine-tuning (optional)
Unfreeze the last few VGG16 layers
base.trainable = True

for layer in base.layers[:-4]: layer.trainable = False

model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.00001), loss="sparse_categorical_crossentropy", metrics=["accuracy"]
)

Train again for fine-tuning
history2 = model.fit(train, validation_data=val, epochs=1
)

Final evaluation
loss, accuracy = model.evaluate(val)
print("Fine-tuned Accuracy:", accuracy)...

ChatGPT

VGG16	Loads the pre-trained VGG16 architecture
weights="imagenet"	Loads weights learned from ImageNet
include_top=False	Removes the original classification layers
input_shape=(128,128,3)	Accepts 128 × 128 RGB images

You

5B. Object detection using Transfer Learning of CNN architectures for the Intel Image Classification Dataset using the below steps: a. Set up the environment b. Loading and preprocessing data c. Choose the pre-trained Model (Transfer Learning) d. Freeze the base layers and add custom layers for fine tuning e. Object detection training f. Compute loss g. Evaluate the Model h. Visualize the prediction i. Fine Tune the Model: Fine-tuning involves unfreezing the top layers of the pre

trained model and retraining it on the new dataset to adapt it better to your task.

```
# Practical 5B: Transfer Learning using MobileNetV2
# Intel Image Classification Dataset (Local Dataset)

# STEP 1: Import libraries
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras import layers, models
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input

# STEP 2: Set local dataset path
dataset_path = "Intel Image Classification"

IMAGE_SIZE = (160, 160)
BATCH_SIZE = 32

# STEP 3: Load local training dataset (80%)
train_data = tf.keras.utils.image_dataset_from_directory( dataset_path,
image_size=IMAGE_SIZE, batch_size=BATCH_SIZE, validation_split=0.2, subset="training",
seed=123
)

# STEP 4: Load local testing/validation dataset (20%)
test_data = tf.keras.utils.image_dataset_from_directory( dataset_path, image_size=IMAGE_SIZE,
batch_size=BATCH_SIZE, validation_split=0.2, subset="validation", seed=123
)

class_names = train_data.class_names
print("Classes:", class_names)

# STEP 5: Preprocess images for MobileNetV2
train_data = train_data.map( lambda x, y: (preprocess_input(x), y)
)

test_data = test_data.map( lambda x, y: (preprocess_input(x), y)
)

# STEP 6: Load pretrained MobileNetV2
base_model = MobileNetV2( weights="imagenet", include_top=False, input_shape=(160, 160, 3)
)
```

```
# Freeze pretrained layers
base_model.trainable = False

# STEP 7: Add custom classification layers
model = models.Sequential([ layers.Input(shape=(160, 160, 3)), base_model,
layers.GlobalAveragePooling2D(), layers.Dense(128, activation="relu"), layers.Dropout(0.3),
layers.Dense(len(class_names), activation="softmax")
])

# STEP 8: Compile model
model.compile( optimizer="adam", loss="sparse_categorical_crossentropy",
metrics=["accuracy"]
)

model.summary()

# STEP 9: Train model
history = model.fit( train_data, validation_data=test_data, epochs=5
)

# STEP 10: Evaluate model
loss, accuracy = model.evaluate(test_data)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy)

# STEP 11: Fine-tuning
base_model.trainable = True

# Freeze all layers except last 30
for layer in base_model.layers[:-30]: layer.trainable = False

# Keep Batch Normalization layers frozen
for layer in base_model.layers: if isinstance(layer, layers.BatchNormalization): layer.trainable =
False

# Compile with a small learning rate
model.compile( optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
loss="sparse_categorical_crossentropy", metrics=["accuracy"]
)

# Train fine-tuned model
fine_history = model.fit( train_data, validation_data=test_data, epochs=3
)
```

```
# STEP 12: Evaluate fine-tuned model
loss, accuracy = model.evaluate(test_data)

print("Fine-tuned Loss:", loss)
print("Fine-tuned Accuracy:", accuracy)

# STEP 13: Plot accuracy
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")

plt.plot(np.arange(5, 8), fine_history.history["accuracy"], label="Fine-tuning Accuracy")
plt.plot(np.arange(5, 8), fine_history.history["val_accuracy"], label="Fine-tuning Validation
Accuracy")

plt.title("Model Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

# STEP 14: Plot loss
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.plot(np.arange(5, 8), fine_history.history["loss"], label="Fine-tuning Loss")
plt.plot(np.arange(5, 8), fine_history.history["val_loss"], label="Fine-tuning Validation Loss")

plt.title("Model Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

# STEP 15: Visualize predictions
for images, labels in test_data.take(1):

predictions = model.predict(images) predicted_labels = np.argmax(predictions, axis=1)

plt.figure(figsize=(12, 6))

for i in range(6): plt.subplot(2, 3, i + 1)

# Convert image back for display display_image = (images[i].numpy() + 1) * 127.5
display_image = np.clip(display_image, 0, 255).astype("uint8")
```

plt.imshow(display_image)

actual = class_names[labels[i].numpy()] predicted = class_names[predicted_labels[i]]

plt.title("Actual: " + actual + "\nPredicted: " + predicted) plt.axis("off")

plt.tight_layout() plt.show()

STEP 16: Save model
model.save("intel_transfer_learning_model.keras")

print("Model saved successfully!")
5B. Object detection using Transfer Learning of CNN architectures for the Intel Image Classification Dataset using the below steps: a. Set up the environment b. Loading and preprocessing data c. Choose the pre-trained Model (Transfer Learning) d. Freeze the base layers and add custom layers for fine tuning e. Object detection training f. Compute loss g. Evaluate the Model h. Visualize the prediction i. Fine Tune the Model: Fine-tuning involves unfreezing the top layers of the pre
trained model and retraining it on the new dataset to adapt it better to your task....

ChatGPT

dataset_path	Dataset folder
image_size	Resizes images to 160 × 160
batch_size	Processes 32 images at a time
validation_split=0.2	Reserves 20% for validation
subset="training"	Loads 80% training images
seed=123	Keeps the split consistent